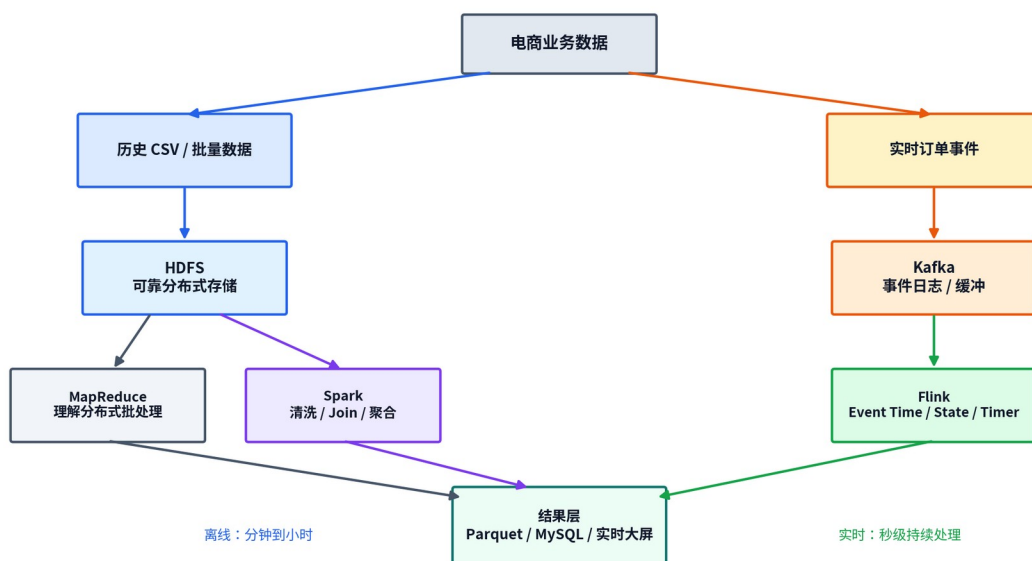


《大数据综合项目实战》

第 1 讲 · 项目需求、总体架构与数据设计

目标：把 HDFS、YARN、MapReduce、Spark、Kafka、Flink 从“分别会一点”变成“能完成一个完整数据系统”。

综合项目总体架构：离线链 + 实时链



本讲原则：先设计业务和数据，再碰框架。你今天的目标不是“把所有服务都启动”，而是把项目地图、数据字段、目录和验收标准建立起来。

1. 为什么现在要做综合项目

你已经分别学习了 HDFS、MapReduce、Spark、Kafka 和 Flink。单个知识点已经不少，但真正做项目时，难点通常不是“某个 API 不会”，而是下面这些系统级问题：

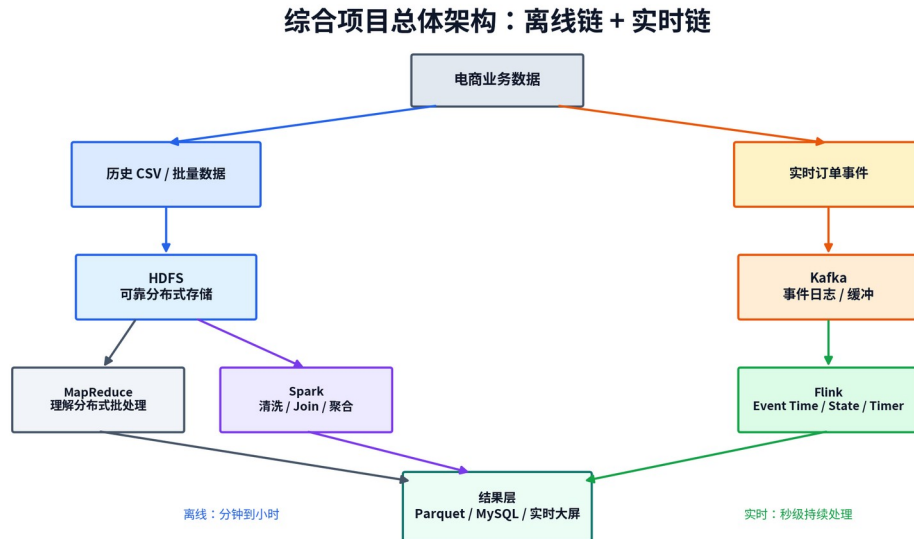
- 数据到底应该放 HDFS 还是 Kafka？
- 同一个需求，什么时候用 MapReduce，什么时候用 Spark？
- 哪些统计是离线批处理，哪些必须实时计算？
- Flink 的 Watermark、Window、State、Timer 在真实业务里怎么同时出现？
- 出现结果不对、Lag 增长、Backpressure 或 Checkpoint 失败时，先查哪一层？

最低闭环能力：给你一个新需求，你能自己画架构、设计数据、选择组件、写出最小程序、运行、验证结果，并能根据证据定位错误。

1.1 这个综合项目最终要做什么

项目名称：电商大数据离线 + 实时分析平台。

我们会同时构建两条链：一条处理历史数据，一条处理正在发生的实时订单事件。最终让两条链在结果层汇合。



1.2 本项目不会做什么

- 不追求漂亮前端大屏；大屏只作为“结果展示层”的概念。
- 不在第一阶段引入复杂微服务、Docker、Kubernetes、湖仓等新工具。
- 不为了“技术栈齐全”硬塞组件；每个组件必须有明确职责。
- 不把“Job 显示 SUCCESS”当作成功；每个阶段都要有手算标准答案和检查证据。

2. 业务场景：一家电商平台到底要分析什么

假设我们运营一个小型电商平台。业务每天产生用户、商品、订单和订单状态变化。系统需要同时回答“历史上发生了什么”和“现在正在发生什么”。

2.1 离线问题：对已经沉淀的数据做分析

业务问题	典型输出	后续主要工具
每个城市昨天销售额是多少？	Beijing = 6874	MapReduce / Spark
每个商品卖了多少件？	P002 = 5 件	Spark
销售额最高的 Top-3 商品是什么？	商品排名	Spark
用户和订单、商品表怎样关联？	宽表 / 分析表	Spark Join
历史结果如何高效保存？	Parquet 文件	Spark + HDFS

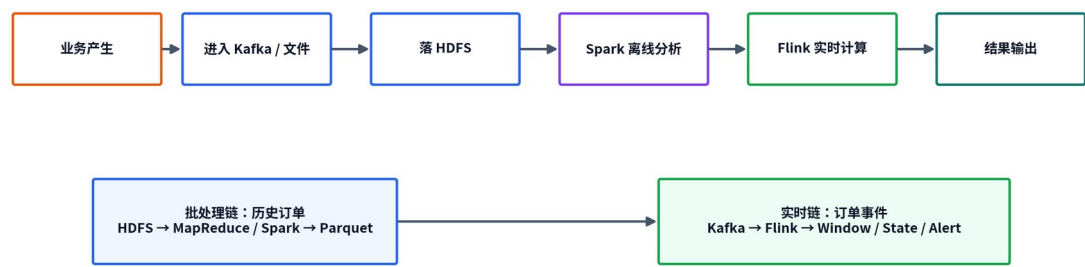
2.2 实时问题：对永远不会结束的事件流持续计算

业务问题	需要的 Flink 概念
每分钟各城市实时销售额	Event Time + Watermark + Window
订单创建 15 分钟仍未支付报警	Keyed State + Timer
实时结果如何不断更新	Dynamic Table / Changelog / Sink
TaskManager 挂掉后怎样恢复	Checkpoint + Source Offset
Kafka Lag 一直增加怎么办	Backpressure + Web UI + Sink 吞吐

关键区别：离线分析面对的是“有限数据集”；实时分析面对的是“数据永远继续到来”。这会直接改变计算模型。

3. 系统架构：每个组件到底负责什么

一条数据在综合项目中的生命周期



核心：不是“工具串起来”，而是让每种工具解决它最擅长的问题。

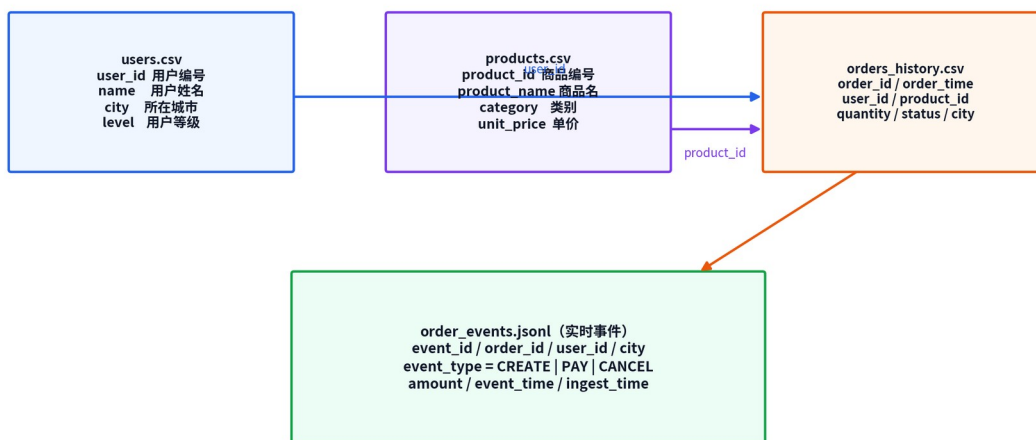
图 2 数据在项目中的生命周期

组件	在本项目中的职责	不要把它误解成
HDFS	保存历史 CSV、Parquet、Checkpoint 等大文件	数据库或消息队列
YARN	给 MapReduce / Spark 等应用分配集群资源	计算框架本身
MapReduce	做一个经典离线基线任务，巩固 Shuffle / Reduce	实时计算框架
Spark	历史数据清洗、Join、聚合、Top-N、Parquet	消息队列
Kafka	保存并传递实时订单事件，提供 Offset / Consumer Group	实时计算引擎
Flink	持续处理 Kafka 事件，管理 Event Time、State、Timer	长期存储系统
MySQL / 文件	保存最终可查询的业务结果	核心计算引擎

4. 数据设计：项目真正从这里开始

原则：先把“字段是什么意思、谁关联谁、什么是主键、什么是事件时间”讲清楚，再写任何 MapReduce / Spark / Flink 代码。

项目数据模型：三张离线表 + 一条实时事件流



离线表回答“过去发生了什么”；实时事件流回答“现在正在发生什么”。

图 3 项目数据模型

4.1 users.csv：用户维表

字段	英文全称/含义	现实含义	示例
user_id	User ID / 用户编号	唯一识别一个用户	U001
name	Name / 姓名	本项目只用于演示关联	ZhangSan
city	City / 城市	用户常驻城市	Beijing
level	Level / 用户等级	VIP 或 Normal	VIP

```

user_id,name,city,level
U001,ZhangSan,Beijing,VIP
U002,LiSi,Shanghai,Normal
  
```

4.2 products.csv：商品维表

字段	含义	为什么需要
product_id	商品编号	订单通过它关联商品
product_name	商品名称	展示与解释
category	商品类别	以后做类别聚合
unit_price	单价	和 quantity 相乘得到订单金额

4.3 orders_history.csv：历史订单事实表

字段	含义	例子/说明
order_id	订单编号	O1001，唯一订单
order_time	订单时间	历史批处理用的业务时间
user_id	用户编号	关联 users.csv
product_id	商品编号	关联 products.csv
quantity	购买数量	必须是正整数
status	订单状态	PAID / CANCELLED / REFUNDED
city	下单城市	后续按城市聚合

业务规则：离线销售额只统计 status = PAID 的订单；CANCELLED 和 REFUNDED 不计入销售额。

5. 第一批标准数据与“手算答案”

项目 Starter Pack 里已经准备好 12 条历史订单。你必须先会手算，因为以后 Spark/Flink 的结果只有和标准答案一致，才算业务正确。

order_id	city	product	qty	status	金额
O1001	Beijing	P002	2	PAID	2×129=258
O1002	Shanghai	P001	1	PAID	5999
O1003	Guangzhou	P003	3	PAID	177
O1004	Beijing	P004	1	CANCELLED	不计
O1005	Beijing	P005	1	PAID	499
O1006	Shenzhen	P002	1	REFUNDED	不计
O1007	Shanghai	P004	2	PAID	598
O1008	Guangzhou	P005	1	PAID	499
O1009	Beijing	P003	2	PAID	118
O1010	Beijing	P001	1	PAID	5999
O1011	Shenzhen	P004	1	PAID	299
O1012	Shanghai	P002	3	PAID	387

5.1 本项目第一组标准答案

指标	标准答案
PAID 订单数	10
Beijing 销售额	6874
Shanghai 销售额	6984
Guangzhou 销售额	676
Shenzhen 销售额	299
总销售额	14833

指标	标准答案

以后每一阶段都这样验收：程序“能跑”只是技术正确；结果与标准答案一致，才是业务正确。

6. 实时事件为什么不能继续沿用“订单表”思维

历史订单表是一行一个订单的最终状态；实时系统看到的却是一个订单在不同时间发生的事件。比如同一个订单先 CREATE，再 PAY，或者 CREATE 后 CANCEL。

```
{"event_id":"E001","order_id":"02001","user_id":"U001","city":"Beijing",  
  "amount":258.0,"event_type":"CREATE","event_time":"2026-09-29 10:00:05"}
```

```
{"event_id":"E003","order_id":"02001","user_id":"U001","city":"Beijing",  
  "amount":258.0,"event_type":"PAY","event_time":"2026-09-29 10:01:10"}
```

后面为什么需要 State：Flink 收到 CREATE 后不能忘记它；如果稍后收到 PAY，需要把两个事件联系起来。如果 15 分钟都没有 PAY，则 Timer 触发告警。

7. 项目目录：从第一天就养成工程习惯

建议整个综合项目统一放在 Ubuntu 虚拟机的用户目录下。不要今天把文件放 Desktop，明天放 /tmp，后天又放 HDFS 根目录。

```
~/bigdata-capstone/
├── data/
│   ├── users.csv
│   ├── products.csv
│   ├── orders_history.csv
│   └── order_events.jsonl
├── mapreduce/
├── spark/
├── kafka/
├── flink/
├── sql/
├── scripts/
└── output/
```

7.1 本地路径、HDFS 路径、Kafka Topic 必须分清

类型	例子	谁负责
Ubuntu 本地文件	~/bigdata-capstone/data/orders_history.csv	Linux 文件系统
HDFS 路径	/bigdata-capstone/raw/orders_history.csv	HDFS
Kafka Topic	orders_raw	Kafka Broker
Spark/Flink 输出	/bigdata-capstone/output/...	取决于 Sink

高频错误：“No such file”不一定是文件不存在，也可能是你把本地路径、HDFS 路径和 Kafka Topic 当成同一种地址了。

8. 环境约定：今天先检查，不急著全部启动

这个综合项目会跨多个系统。如果第一讲就同时启动 HDFS、YARN、Kafka、Spark、Flink、MySQL，很容易把学习变成“服务为什么启动不了”。所以今天只做环境盘点和项目目录准备。

组件	本讲要求	后面什么时候真正使用
Ubuntu VM	能启动、能 SSH	每一讲
VS Code Remote SSH	能连接 VM、打开远程目录	每一讲
Hadoop / HDFS / YARN	确认已有，不要求今天跑 Job	第 2-3 讲
Spark	确认命令存在	第 4-5 讲
Kafka	确认目录/命令存在	第 6 讲
Flink	确认已有可用 Java 环境	第 7-9 讲
MySQL	可选结果层	后期

8.1 保姆式操作：从 Windows 到 Ubuntu 项目目录

- 1. 启动 Ubuntu 虚拟机，等待登录界面和网络初始化完成。
- 2. 在 Windows 打开 VS Code。
- 3. 点击左下角远程连接状态，使用 Remote SSH 连接你的 Hadoop/Ubuntu 虚拟机。
- 4. 确认 VS Code 左下角显示 SSH 远程主机，而不是本地 Windows。
- 5. 在 VS Code 菜单选择 File → Open Folder，打开 /home/hadoop/bigdata-capstone（如果你的用户名不是 hadoop，就替换为自己的用户名）。
- 6. 打开 Terminal，确认提示符是 Ubuntu 远程终端。执行 pwd，确保当前位置在远程 Linux，而不是 Windows。

```
pwd
whoami
hostname
java -version
hadoop version 2>/dev/null | head
python3 --version
```

判断标准：看到 Linux 路径（例如 /home/hadoop/...）才说明你正在操作虚拟机。Windows 的 C:\Users\... 不是 HDFS，也不是 Ubuntu。

9. 本讲动手任务：建立项目骨架

9.1 创建目录

```
cd ~
mkdir -p bigdata-capstone/{data,mapreduce,spark,kafka,flink,sql,scripts,output}
cd ~/bigdata-capstone
pwd
ls -al
```

你应该看到 data、mapreduce、spark、kafka、flink、sql、scripts、output 八个目录。

9.2 把 Starter Pack 放进虚拟机

我同时提供了一个 Starter Pack ZIP。最简单的方法是把 ZIP 拖入 VS Code 的远程文件资源管理器，然后在远程终端解压。

```
cd ~/bigdata-capstone
unzip BigData_Integrated_Project_Lecture1_Starter.zip
find . -maxdepth 2 -type f | sort
```

如果你不想拖文件：也可以在 Windows 先解压，再通过 VS Code 复制整个 data/ 和 scripts/ 目录到远程项目目录。

9.3 运行数据校验脚本

```
cd ~/bigdata-capstone
python3 scripts/01_verify_data.py
```

正确输出中必须出现：

```
PAID 订单数 = 10
Beijing: 6874
Guangzhou: 676
Shanghai: 6984
Shenzhen: 299
总销售额 = 14833
实时事件条数 = 8
[PASS] 数据包结构与标准答案校验通过
```

如果结果不一致：先不要进入 HDFS。第一讲就把数据文件和标准答案对齐；否则后面所有框架算出来的结

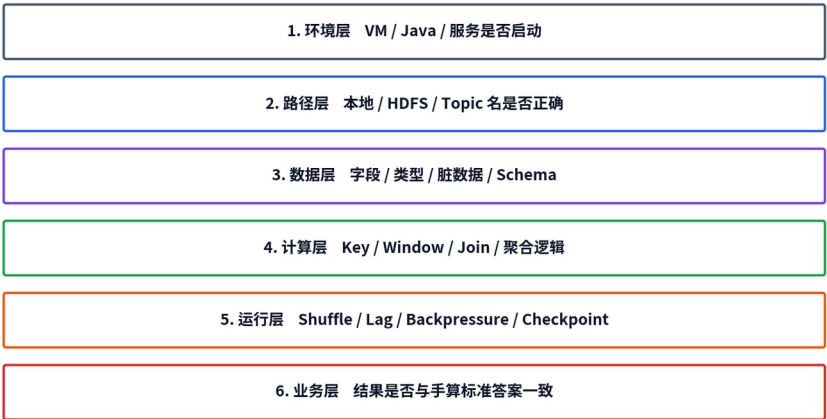
果都会被错误数据污染。

10. 为什么项目必须保留“证据”

以后每一个阶段都要保存四类证据。这样你遇到错误时不是凭感觉，而是能证明哪一层正常、哪一层异常。

证据	例子	能证明什么
输入证据	head / hdfs dfs -cat / Kafka consumer	输入确实存在
执行证据	YARN / Spark UI / Flink UI	任务确实执行
结果证据	part-* / Parquet / Sink 输出	程序确实产出结果
业务证据	手算答案 / 行数 / 汇总值	结果逻辑确实正确

综合项目统一排错顺序

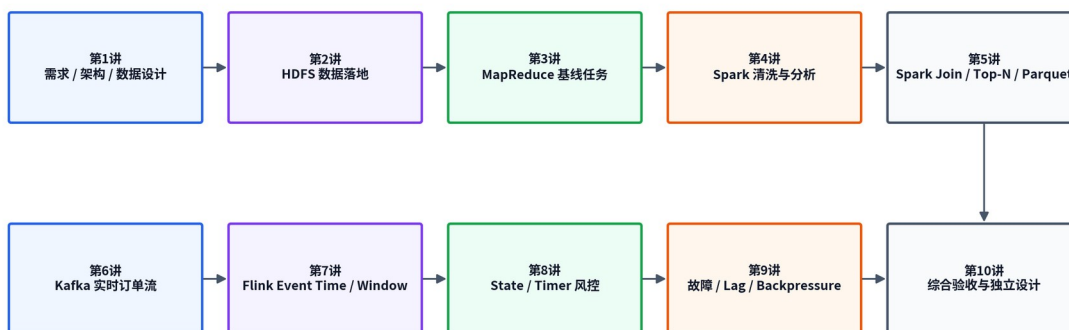


原则：先定位“哪一层出错”，再修改；不要一看到失败就重装软件或调大内存。

图 4 整个项目统一排错顺序

11. 10 讲综合项目路线：每一讲到底要产出什么

综合项目 10 讲路线图



最终目标：给你一个新需求，你能自己判断数据放哪、用谁算、Key/Window/State 怎么设计、出了问题先查哪里。

图 5 综合项目路线图

讲次	任务	最低产出
1	需求、架构、数据设计	项目目录 + 数据包 + 标准答案
2	HDFS 数据落地	raw 数据进入 HDFS，并验证 Block/路径
3	MapReduce 基线	城市销售额 MapReduce Job
4	Spark 清洗与 DataFrame	清洗后的标准 DataFrame
5	Spark Join/Top-N/Parquet	分析结果 + Parquet
6	Kafka 实时订单流	Topic + Producer + Consumer + Lag
7	Flink 窗口实时统计	每分钟城市销售额
8	State + Timer 风控	15 分钟未支付报警
9	故障与性能实验	Checkpoint 恢复 + Lag/Backpressure 诊断
10	最终综合验收	独立设计一个新需求

12. 今天完成到什么程度才算“第 1 讲学完”

- 你能用一句话说明 HDFS、Spark、Kafka、Flink 在项目中分别干什么。
- 你能解释 orders_history.csv 与 order_events.jsonl 为什么不是同一种数据。
- 你能说出 user_id、product_id、order_id 各自用于关联谁。
- 你已经在 Ubuntu 中建立 ~/bigdata-capstone/ 项目目录。
- 你已经把 Starter Pack 放进项目目录。
- 你运行 01_verify_data.py 后得到标准答案 14833。
- 你能分清本地 Linux 路径、HDFS 路径和 Kafka Topic。
- 你知道出现错误时先定位“环境/路径/数据/计算/运行/业务”哪一层，而不是先重装。

本讲结束条件：上面 8 项全部能做到，再进入第 2 讲。综合项目不追求赶进度，追求每一层都能独立验收。

13. 常见疑问与误区

疑问/误区	解释
既然 Spark 更方便，为什么项目还要做一次 MapReduce？	因为你需要把 Shuffle、Partition、Reduce 等底层机制和真实任务对应一次。
Kafka 能不能直接替代 HDFS？	不能。Kafka 适合事件日志和流式传输；HDFS 适合长期大文件存储。
Flink 能不能把所有离线任务也做了？	技术上很多可以，但项目要按工具优势分工，不追求“一种框架做全部”。
为什么数据集这么小？	教学阶段小数据用于手算和快速验证；原理与大数据集相同，后面再扩量。
今天为什么不启动所有服务？	先建立数据与架构心智模型。一次启动太多系统会把注意力浪费在环境故障上。
城市字段为什么订单表里也有，users 表里也有？	故意保留现实中常见的冗余字段；后面会讨论“下单城市”和“用户常驻城市”不是同一语义。

14. 练习题

1. 如果需求改成“统计每个商品的总销量”，MapReduce/Spark 的聚合 Key 应该是什么？
2. 如果要求“实时检测订单创建 15 分钟未支付”，为什么只用 15 分钟窗口不够直观？
3. 历史订单文件应该放 HDFS 还是 Kafka？实时 CREATE/PAY 事件应该放哪里？

4. 如果 `python3 scripts/01_verify_data.py` 报 `users.csv` 不存在，你应该先查环境层、路径层还是业务层？
5. 如果程序 SUCCESS，但总销售额算成 14962，你应该把它判定为成功吗？为什么？

参考答案

1. `product_id`。因为需要让同一商品的订单在聚合阶段见面。
2. 未支付超时是“每个订单自己的截止时间”，更自然的模型是 `order_id Keyed State + Timer`，而不是把所有订单硬塞进统一窗口。
3. 历史批量文件放 HDFS；持续到来的实时订单事件放 Kafka。
4. 先查路径层：当前工作目录、Starter Pack 是否解压到正确位置、文件名是否一致。
5. 不能。技术执行成功不代表业务正确，必须与标准答案 14833 对齐。

15. 下一讲预告：真正把数据放进 HDFS

第 2 讲我们才正式启动 HDFS / YARN，并把 `orders_history.csv`、`users.csv`、`products.csv` 放入统一的 HDFS 项目目录。然后用命令逐步验证：文件是否真的在 HDFS、Block 如何存储、为什么 Spark/MapReduce 能看到这些数据。

下一讲主线：启动前检查 → 启动 HDFS/YARN → 健康验证 → 创建 HDFS 项目目录 → 上传数据 → `cat / ls / du / fsck` 验证 → 停止服务与清理。